# Table of Contents

**Introduction** ..... **2**

**Dynamic Carousel** ..... **3**

**Add Static Client-Side Files and Client-Side library** ..... **4**

*Add Static Asset Folders and Images* ..... *4*

*Add Client-Side Libraries (Bootstrap and jQuery) using LibMan* ..... *4*

*Reference Scripts and Styles in _Layout.cshtml* ..... *5*

**Configure Static Asset Routing in `Program.cs`** ..... **6**

# Demo: Shop MVC 3, dynamic content, library manager, client-side files

Author: Baifan Zhou
Some explanations are created with the help of ChatGPT and Gemini.

## Introduction

This demo will walk you through the process of adding a dynamic carousel to your Home Page, managing static client-side files, integrating client-side libraries like Bootstrap and jQuery, and configuring middleware for serving static files.

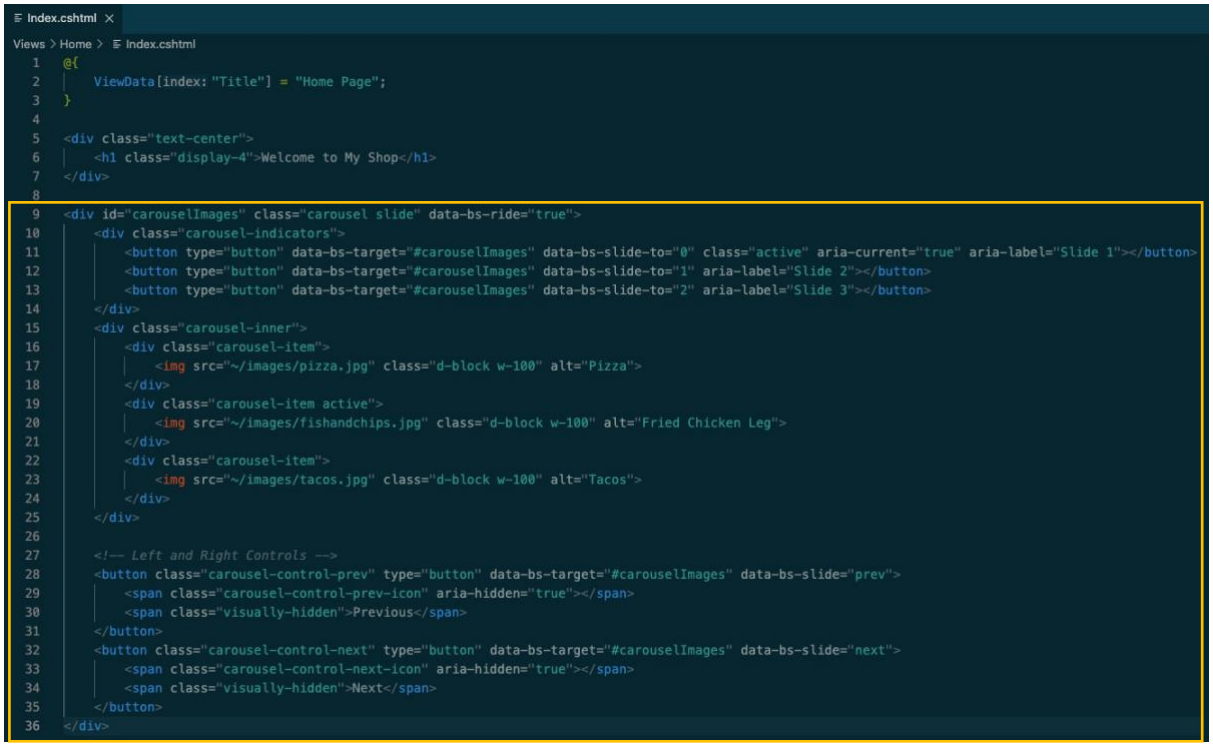
This demo is designed to help you integrate dynamic features into your ASP.NET Core MVC application. You will learn how to:

1. **Add a Dynamic Carousel**:
    - Enhance your Home Page by incorporating a carousel that displays multiple images, creating a more engaging user experience.
2. **Add Static Client-Side Files**:
    - Serve static files like CSS and JavaScript from the `wwwroot` folder, following best practices for structuring your ASP.NET Core MVC application.
    - Create the necessary folders and add images to be displayed in the carousel.
3. **Client-side Library**:
    - Use Library Manager (LibMan) to manage client-side libraries such as Bootstrap and jQuery.
    - libman.json lists the client-side packages, specified versions, and target destinations (wwwroot/lib/...) required by your project.
    - Add the `Microsoft.Web.LibraryManager.Build` package to automatically restore client-side libraries into `wwwroot/lib/` during `dotnet build` or `dotnet run`.
4. **Add Middleware for Static Files**:
    - Configure your application to serve static files using `app.MapStaticAssets()`.
    - Update your layout file (`_Layout.cshtml`) to include references to Bootstrap and jQuery, enhancing the functionality and appearance of your web pages.

By following the steps in this demo, you'll learn how to effectively integrate dynamic content and client-side libraries into your ASP.NET Core MVC application, resulting in a more dynamic and user-friendly web application. Happy coding!

## Dynamic Carousel

Now we want to add some dynamics to our Home Page.
Add the division code into the Index.cshtml file.



```
Index.cshtml
Views > Home > Index.cshtml
@{
    ViewData[index: "Title"] = "Home Page";
}

<div class="text-center">
    <h1 class="display-4">Welcome to My Shop</h1>
</div>

<div id="carouselImages" class="carousel slide" data-bs-ride="true">
    <div class="carousel-indicators">
        <button type="button" data-bs-target="#carouselImages" data-bs-slide-to="0" class="active" aria-current="true" aria-label="Slide 1"></button>
        <button type="button" data-bs-target="#carouselImages" data-bs-slide-to="1" aria-label="Slide 2"></button>
        <button type="button" data-bs-target="#carouselImages" data-bs-slide-to="2" aria-label="Slide 3"></button>
    </div>
    <div class="carousel-inner">
        <div class="carousel-item">
            <img src="~/images/pizza.jpg" class="d-block w-100" alt="Pizza">
        </div>
        <div class="carousel-item active">
            <img src="~/images/fishandchips.jpg" class="d-block w-100" alt="Fried Chicken Leg">
        </div>
        <div class="carousel-item">
            <img src="~/images/tacos.jpg" class="d-block w-100" alt="Tacos">
        </div>
    </div>

    <!-- Left and Right Controls -->
    <button class="carousel-control-prev" type="button" data-bs-target="#carouselImages" data-bs-slide="prev">
        <span class="carousel-control-prev-icon" aria-hidden="true"></span>
        <span class="visually-hidden">Previous</span>
    </button>
    <button class="carousel-control-next" type="button" data-bs-target="#carouselImages" data-bs-slide="next">
        <span class="carousel-control-next-icon" aria-hidden="true"></span>
        <span class="visually-hidden">Next</span>
    </button>
</div>
```

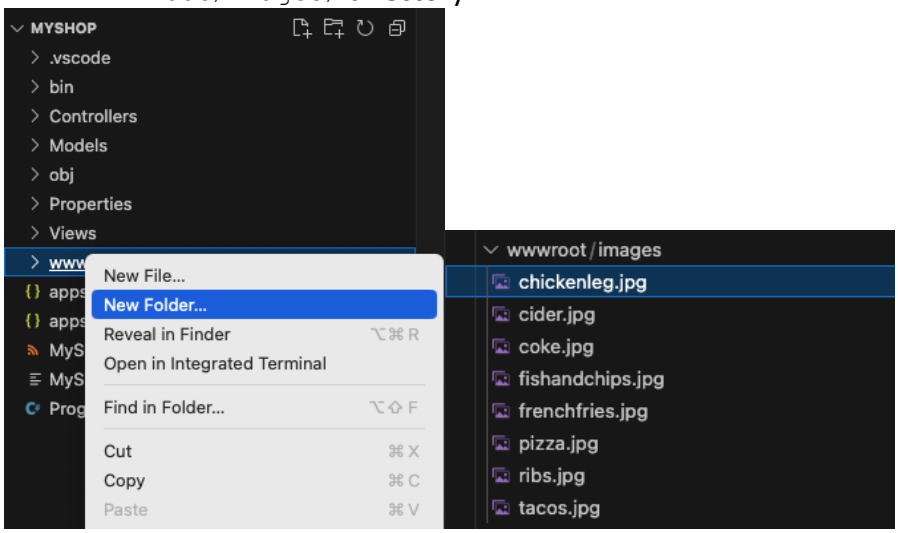
This will add a carousel that displays three images, and left and right arrows control.
To make this work, we need to add some client-side files and JavaScript libraries.

# Add Static Client-Side Files and Client-Side library

In a typical ASP.NET Core MVC application, static files like CSS and JavaScript should be served from the `wwwroot` folder.

## Add Static Asset Folders and Images

- Ensure a `wwwroot` folder exists in your project's root directory.
- Inside `wwwroot`, create a subfolder named `images` (Note: ensure all folder and file names use lowercase to remain fully cross-platform compatible across macOS, Linux, and Windows).
- Copy your project images (e.g., `pizza.jpg`, `tacos.jpg`, `fishandchips.jpg`) into the `wwwroot/images/` directory.

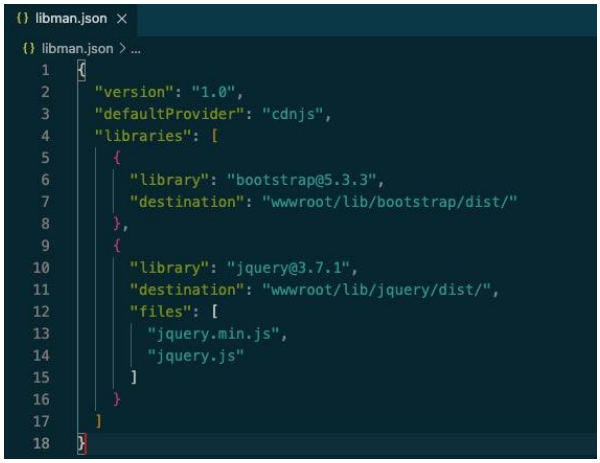



## Add Client-Side Libraries (Bootstrap and jQuery) using LibMan

ASP.NET Core uses **Library Manager (LibMan)** configured via a manifest file (`libman.json`).

**1. Create `libman.json` in the Project Root**

In the root directory of your project (alongside your `.csproj` file), create a new file named `libman.json` and paste the following configuration. This configuration specifies the exact package versions and their output destinations inside `wwwroot/lib`. The version numbers guarantee that your code has the identical dependencies when running on different machines.



**2. Enable Automatic Library Restoration on Build**
To ensure LibMan automatically downloads these files into `wwwroot/lib` whenever you build or run your project, add the LibMan build package to your project.
Open your integrated terminal in VS Code and run:

```
dotnet add package Microsoft.Web.LibraryManager.Build
```

Now, every time you execute `dotnet build` or `dotnet run`, .NET automatically restores the specified Bootstrap and jQuery files into `wwwroot/lib` without requiring any manual CLI commands.

**3. Update `.gitignore`**
Add the downloaded library folder to your `.gitignore` file so vendor assets are not committed to Git, use one of the below options:

- `wwwroot/lib/`
  - Matches `wwwroot/lib/` located at the root level of your Git repository.
- `**/wwwroot/lib/`
  - Matches `wwwroot/lib/` in any depth.

## Reference Scripts and Styles in _Layout.cshtml

Add the CSS and JavaScript references to your `Views/Shared/_Layout.cshtml` file. Note that we use **`bootstrap.bundle.min.js`** because it includes Popper.js, which is required for interactive elements like dropdowns and carousels.
In `<head>`:

```
<link rel="stylesheet" href="~/lib/bootstrap/dist/css/bootstrap.min.css" />
<script src="~/lib/jquery/dist/jquery.min.js" defer></script>
<script src="~/lib/bootstrap/dist/js/bootstrap.bundle.min.js"
defer></script>
```

_Layout.cshtml

Views > Shared > _Layout.cshtml

```
1   <!DOCTYPE html>
2   <html lang="en">
3   <head>
4       <meta charset="utf-8" />
5       <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6       <title>@ViewData[index: "Title"] - My Shop</title>
7       <link rel="stylesheet" href="~/lib/bootstrap/dist/css/bootstrap.min.css" />
8       <script src="~/lib/jquery/dist/jquery.min.js" defer></script>
9       <script src="~/lib/bootstrap/dist/js/bootstrap.bundle.min.js" defer></script>
10  </head>
```

The `~` character represents the root of static assets (`wwwroot`). Using ASP.NET Core's `href="~/..."` or `src="~/..."` tag helpers allows .NET to automatically apply asset fingerprinting for browser cache management.

What `defer` does: It downloads the script in the background while HTML continues parsing, and executes the script only after the DOM is fully parsed. This gives you the speed benefits of placing scripts at the bottom of `<body>`.

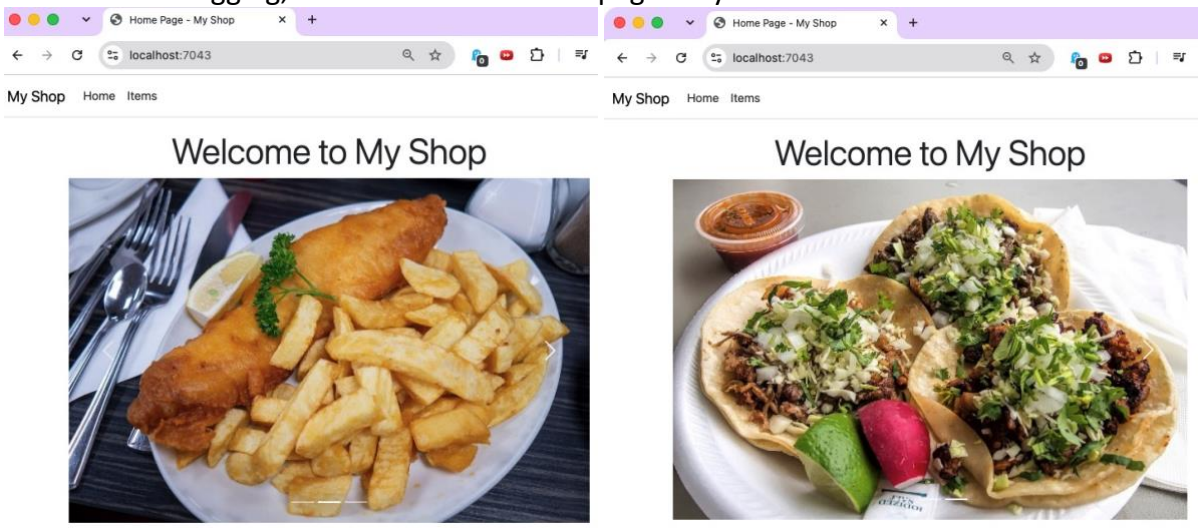
## Configure Static Asset Routing in `Program.cs`

In the Program.cs, add a middleware of app.UseStaticFiles (Line12), this allows using the files in the folder wwwroot.

In modern ASP.NET Core, the legacy `app.UseStaticFiles()` middleware is replaced with `app.MapStaticAssets()`. This endpoint routing method automatically handles file compression (Gzip/Brotli) and cache-busting fingerprinting during application build.

```
Program.cs
Program.cs
1   var builder = WebApplication.CreateBuilder(args);
2
3   builder.Services.AddControllersWithViews();
4
5   var app = builder.Build();
6
7   if (app.Environment.IsDevelopment())
8   {
9       app.UseDeveloperExceptionPage();
10  }
11
12  app.MapStaticAssets(); // Enable static assets from wwwroot (images, JS, CSS)
13
14  app.MapDefaultControllerRoute();
15
16  // app.MapControllerRoute(
17  //     name: "default",
18  //     pattern: "{controller=Home}/{action=Index}/{id?}");
19
20  app.Run();
```

Now Start Debugging, and see that the home page is dynamic:




Congratulations! ☺